With ordinary integer arithmetic, the numerator is likely to overflow; and, unfortunately, an expression like `jdif/(ntot/n)` is not equivalent. We therefore need to use double-length integers, type `Ullong`, usually 64 bits, just for this operation.

The internally set variable `minint`, which is the minimum allowed number of discrete steps between the upper and lower bounds, determines when new low-significance digits are added. `minint` must be large enough to provide resolution of all the input characters. That is, we must have $p_i \times \texttt{minint} > 1$ for all i. A value of $100N_{\rm ch}$, or $1.1/\min p_i$, whichever is larger, is generally adequate. However, for safety, the routine takes `minint` to be as large as possible, with the product `minint*nradd` just smaller than overflow. This results in some time inefficiency, and in a few unnecessary characters being output at the end of a message. You can decrease `minint` if you want to live closer to the edge.

If radix-changing, rather than compression, is your primary aim (for example to convert an arbitrary file into printable characters), then you are of course free to set all the components of `nfreq` equal, say, to 1.

While the output radix is limited to 256 (so that values fit into a byte), the input alphabet size $N_{\rm ch} =$ `nch` can be less than, equal to, or greater than 256.

CITED REFERENCES AND FURTHER READING:

Sayood, K. 2005, *Introduction to Data Compression*, 3rd ed. (San Francisco: Morgan Kaufmann).

Salomon, D. 2004, *Data Compression: The Complete Reference*, 3rd ed. (New York: Springer).

Wayner, P. 1999, *Compression Algorithms for Real Programmers* (San Francisco: Morgan Kaufmann).

Witten, I.H., Neal, R.M., and Cleary, J.G. 1987, "Arithmetic Coding for Data Compression," *Communications of the ACM*, vol. 30, pp. 520–540.[1]

22.7 Arithmetic at Arbitrary Precision

Let's compute the number π to a couple of thousand decimal places. In doing so, we'll learn some things about multiple precision arithmetic on computers and meet quite an unusual application of the fast Fourier transform (FFT). We'll also develop a set of routines that you can use for other calculations at any desired level of arithmetic precision.

To start with, we need an analytic algorithm for π. Useful algorithms are quadratically convergent, i.e., they double the number of significant digits at each iteration. Quadratically convergent algorithms for π are based on the *AGM (arithmetic geometric mean)* method, which also finds application to the calculation of elliptic integrals (cf. §6.12) and in advanced implementations of the ADI method for elliptic partial differential equations (§20.5). Borwein and Borwein [1] treat this subject, which is beyond our scope here. One of their algorithms for π starts with the initializations

$$\begin{aligned} X_0 &= \sqrt{2} \\ \pi_0 &= 2 + \sqrt{2} \\ Y_0 &= \sqrt[4]{2} \end{aligned} \tag{22.7.1}$$

and then, for $i = 0, 1, \ldots,$ repeats the iteration

$$\begin{aligned} X_{i+1} &= \frac{1}{2}\left(\sqrt{X_i} + \frac{1}{\sqrt{X_i}}\right) \\ \pi_{i+1} &= \pi_i \left(\frac{X_{i+1}+1}{Y_i+1}\right) \\ Y_{i+1} &= \frac{Y_i\sqrt{X_{i+1}} + \dfrac{1}{\sqrt{X_{i+1}}}}{Y_i+1} \end{aligned} \tag{22.7.2}$$

The value π emerges as the limit π_∞.

Now to the question of how to do arithmetic to arbitrary precision: In a high-level language like C++, a natural choice is to work in radix (base) 256, so that character arrays can be directly interpreted as strings of digits. At the very end of our calculation, we will want to convert our answer to radix 10, but that is essentially a frill for the benefit of human ears, accustomed to the familiar chant, "three point one four one five nine. . . ." For any less frivolous calculation, we would likely never leave base 256 (or the thence trivially reachable hexadecimal, octal, or binary bases).

We will adopt the convention of storing digit strings in the "human" ordering, that is, with the first stored digit in an array being most significant, the last stored digit being least significant. The opposite convention would, of course, also be possible. "Carries," where we need to partition a number larger than 255 into a low-order byte and a high-order carry, present a minor programming annoyance, solved, in the routines below, by the use of the functions `lobyte` and `hibyte`. It will be our usual convention to assume that the digit strings represent floating-point numbers with the radix point falling after the the first digit. When an operation results in a number that requires more digits in front of the decimal point, it is the responsibility of the user to shift the digits to the right and keep track of any excess factors of 256 that this implies.

It is easy at this point, following Knuth [2], to write a routines for the "fast" arithmetic operations: short addition (adding a single byte to a string), addition, subtraction, short multiplication (multiplying a string by a single byte), short division, ones-complement negation, and a couple of utility operations, copying and left-shifting strings. These are implemented in the following `MParith` object. The additional routines that are declared, but not defined, are discussed below.

mparith.h

```
struct MParith {
```

Multiple precision arithmetic operations done on character strings, interpreted as radix 256 numbers with the radix point after the first digit. Implementations for the simpler operations are listed here.

```
    void mpadd(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```

Adds the unsigned radix 256 numbers `u` and `v`, yielding the unsigned result `w`. To achieve the full available accuracy, the array `w` must be longer, by one element, than the shorter of the two arrays `u` and `v`.

```
        Int j,n=u.size(),m=v.size(),p=w.size();
        Int n_min=MIN(n,m),p_min=MIN(n_min,p-1);
        Uint ireg=0;
        for (j=p_min-1;j>=0;j--) {
            ireg=u[j]+v[j]+hibyte(ireg);
            w[j+1]=lobyte(ireg);
        }
        w[0]=hibyte(ireg);
        if (p > p_min+1)
```

```
        for (j=p_min+1;j<p;j++) w[j]=0;
}

void mpsub(Int &is, VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```

Subtracts the unsigned radix 256 number `v` from `u` yielding the unsigned result `w`. If the result is negative (wraps around), `is` is returned as -1; otherwise it is returned as 0. To achieve the full available accuracy, the array `w` must be as long as the shorter of the two arrays `u` and `v`.

```
    Int j,n=u.size(),m=v.size(),p=w.size();
    Int n_min=MIN(n,m),p_min=MIN(n_min,p-1);
    Uint ireg=256;
    for (j=p_min-1;j>=0;j--) {
        ireg=255+u[j]-v[j]+hibyte(ireg);
        w[j]=lobyte(ireg);
    }
    is=hibyte(ireg)-1;
    if (p > p_min)
        for (j=p_min;j<p;j++) w[j]=0;
}

void mpsad(VecUchar_O &w, VecUchar_I &u, const Int iv) {
```

Short addition: The integer `iv` (in the range $0 \le$ `iv` ≤ 255) is added to the least significant radix position of unsigned radix 256 number `u`, yielding result `w`. To ensure that the result does not require two digits before the radix point, one may first right-shift the operand `u` so that the first digit is 0, and keep track of multiples of 256 separately.

```
    Int j,n=u.size(),p=w.size();
    Uint ireg=256*iv;
    for (j=n-1;j>=0;j--) {
        ireg=u[j]+hibyte(ireg);
        if (j+1 < p) w[j+1]=lobyte(ireg);
    }
    w[0]=hibyte(ireg);
    for (j=n+1;j<p;j++) w[j]=0;
}

void mpsmu(VecUchar_O &w, VecUchar_I &u, const Int iv) {
```

Short multiplication: The unsigned radix 256 number `u` is multiplied by the integer `iv` (in the range $0 \le$ `iv` ≤ 255), yielding result `w`. To ensure that the result does not require two digits before the radix point, one may first right-shift the operand `u` so that the first digit is 0, and keep track of multiples of 256 separately.

```
    Int j,n=u.size(),p=w.size();
    Uint ireg=0;
    for (j=n-1;j>=0;j--) {
        ireg=u[j]*iv+hibyte(ireg);
        if (j < p-1) w[j+1]=lobyte(ireg);
    }
    w[0]=hibyte(ireg);
    for (j=n+1;j<p;j++) w[j]=0;
}

void mpsdv(VecUchar_O &w, VecUchar_I &u, const Int iv, Int &ir) {
```

Short division: The unsigned radix 256 number `u` is divided by the integer `iv` (in the range $0 \le$ `iv` ≤ 255), yielding a quotient `w` and a remainder `ir` (with $0 \le$ `ir` ≤ 255). To achieve the full available accuracy, the array `w` must be as long as the array `u`.

```
    Int i,j,n=u.size(),p=w.size(),p_min=MIN(n,p);
    ir=0;
    for (j=0;j<p_min;j++) {
        i=256*ir+u[j];
        w[j]=Uchar(i/iv);
        ir=i % iv;
    }
    if (p > p_min)
        for (j=p_min;j<p;j++) w[j]=0;
}
```

```
    void mpneg(VecUchar_IO &u) {
    Ones-complement negate the unsigned radix 256 number u.
        Int j,n=u.size();
        Uint ireg=256;
        for (j=n-1;j>=0;j--) {
            ireg=255-u[j]+hibyte(ireg);
            u[j]=lobyte(ireg);
        }
    }

    void mpmov(VecUchar_O &u, VecUchar_I &v) {
    Move the unsigned radix 256 number v into u. To achieve full accuracy, the array v must
    be as long as the array u.
        Int j,n=u.size(),m=v.size(),n_min=MIN(n,m);
        for (j=0;j<n_min;j++) u[j]=v[j];
        if (n > n_min)
            for(j=n_min;j<n-1;j++) u[j]=0;
    }

    void mplsh(VecUchar_IO &u) {
    Left-shift digits of unsigned radix 256 number u. The final element of the array is set to 0.

        Int j,n=u.size();
        for (j=0;j<n-1;j++) u[j]=u[j+1];
        u[n-1]=0;
    }

    Uchar lobyte(Uint x) {return (x & 0xff);}
    Uchar hibyte(Uint x) {return ((x >> 8) & 0xff);}

    The following, more complicated, methods have discussion and implementation below.
    void mpmul(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v);
    void mpinv(VecUchar_O &u, VecUchar_I &v);
    void mpdiv(VecUchar_O &q, VecUchar_O &r, VecUchar_I &u, VecUchar_I &v);
    void mpsqrt(VecUchar_O &w, VecUchar_O &u, VecUchar_I &v);
    void mp2dfr(VecUchar_IO &a, string &s);
    string mppi(const Int np);
};
```

Full multiplication of two strings of digits, if done by the traditional hand method, is not a fast operation: In multiplying two strings of length N, the multiplicand would be short-multiplied in turn by each byte of the multiplier, requiring $O(N^2)$ operations in all. We will see, however, that *all* the arithmetic operations on numbers of length N can in fact be done in $O(N \times \log N \times \log\log N)$ operations.

The trick is to recognize that multiplication is essentially a *convolution* (§13.1) of the digits of the multiplicand and multiplier, followed by some kind of carry operation. Consider, for example, two ways of writing the calculation 456×789:

$$\begin{array}{r} 456 \\ \times\ 789 \\ \hline 4104 \\ 3648 \\ 3192 \\ \hline 359784 \end{array} \qquad\qquad \begin{array}{rrrrrr} & & 4 & 5 & 6 & \\ \times & & 7 & 8 & 9 & \\ \hline & & 36 & 45 & 54 & \\ & 32 & 40 & 48 & & \\ 28 & 35 & 42 & & & \\ \hline 28 & 67 & 118 & 93 & 54 & \\ \hline 3 & 5 & 9 & 7 & 8 & 4 \end{array}$$

The tableau on the left shows the conventional method of multiplication, in which three separate short multiplications of the full multiplicand (by 9, 8, and 7) are added

to obtain the final result. The tableau on the right shows a different method (sometimes taught for mental arithmetic), where the single-digit cross products are all computed (e.g. $8 \times 6 = 48$), then added in columns to obtain an incompletely carried result (here, the list 28, 67, 118, 93, 54). The final step is a single pass from right to left, recording the single least-significant digit and carrying the higher digit or digits into the total to the left (e.g. $93 + 5 = 98$, record the 8, carry 9).

You can see immediately that the column sums in the right-hand method are components of the convolution of the digit strings, for example $118 = 4 \times 9 + 5 \times 8 + 6 \times 7$. In §13.1, we learned how to compute the convolution of two vectors by the fast Fourier transform (FFT): Each vector is FFT'd, the two complex transforms are multiplied, and the result is inverse-FFT'd. Since the transforms are done with floating arithmetic, we need sufficient precision so that the exact integer value of each component of the result is discernible in the presence of roundoff error. We should therefore allow a (conservative) few times $\log_2(\log_2 N)$ bits for roundoff in the FFT. A number of length N bytes in radix 256 can generate convolution components as large as the order of $(256)^2 N$, thus requiring $16 + \log_2 N$ bits of precision for exact storage. If `it` is the number of bits in the floating mantissa (cf. §22.2), we obtain the condition

$$16 + \log_2 N + \text{few} \times \log_2 \log_2 N < \texttt{it} \tag{22.7.3}$$

We see that single precision, say with $\texttt{it} = 24$, is inadequate for any interesting value of N, while double precision, say with $\texttt{it} = 53$, allows N to be greater than 10^6, corresponding to some millions of decimal digits. The use of `Doub` in the routines `realft` (§12.3) and `four1` (§12.2) is therefore a necessity, not merely a convenience, for this application.

mparith.h

```
void MParith::mpmul(VecUchar_O &w, VecUchar_I &u, VecUchar_I &v) {
```
Uses fast Fourier transform to multiply the unsigned radix 256 integers `u[0..n-1]` and `v[0..m-1]`, yielding a product `w[0..n+m-1]`.

```
    const Doub RX=256.0;
    Int j,nn=1,n=u.size(),m=v.size(),p=w.size(),n_max=MAX(m,n);
    Doub cy,t;
    while (nn < n_max) nn <<= 1;
    nn <<= 1;
    VecDoub a(nn,0.0),b(nn,0.0);
    for (j=0;j<n;j++) a[j]=u[j];
    for (j=0;j<m;j++) b[j]=v[j];
    realft(a,1);
    realft(b,1);
    b[0] *= a[0];
    b[1] *= a[1];
    for (j=2;j<nn;j+=2) {
        b[j]=(t=b[j])*a[j]-b[j+1]*a[j+1];
        b[j+1]=t*a[j+1]+b[j+1]*a[j];
    }
    realft(b,-1);
    cy=0.0;
    for (j=nn-1;j>=0;j--) {
        t=b[j]/(nn >> 1)+cy+0.5;
        cy=Uint(t/RX);
        b[j]=t-cy*RX;
    }
    if (cy >= RX) throw("cannot happen in mpmul");
    for (j=0;j<p;j++) w[j]=0;
    w[0]=Uchar(cy);
    for (j=1;j<MIN(n+m,p);j++) w[j]=Uchar(b[j-1]);
}
```

Find the smallest usable power of 2 for the transform.

Move U and V to double precision floating arrays.

Perform the convolution: First, the two Fourier transforms.

Then multiply the complex results (real and imaginary parts).

Then do the inverse Fourier transform.

Make a final pass to do all the carries.

The 0.5 allows for roundoff error.

Copy answer to output.

With multiplication thus a "fast" operation, division is best performed by multiplying the dividend by the reciprocal of the divisor. The reciprocal of a value V is calculated by iteration of Newton's rule,

$$U_{i+1} = U_i(2 - VU_i) \tag{22.7.4}$$

which results in the quadratic convergence of U_∞ to $1/V$, as you can easily prove. (Many historical supercomputers, and some more recent RISC processors, actually use this iteration to perform divisions.) We can now see where the operations count $N \log N \log\log N$, mentioned above, originates: $N \log N$ is in the Fourier transform, with the iteration to converge Newton's rule giving an additional factor of $\log\log N$.

mparith.h

```
void MParith::mpinv(VecUchar_O &u, VecUchar_I &v) {
```

Character string `v[0..m-1]` is interpreted as a radix 256 number with the radix point after (nonzero) `v[0]`; `u[0..n-1]` is set to the most significant digits of its reciprocal, with the radix point after `u[0]`.

```
    const Int MF=4;
    const Doub BI=1.0/256.0;
    Int i,j,n=u.size(),m=v.size(),mm=MIN(MF,m);
    Doub fu,fv=Doub(v[mm-1]);
    VecUchar s(n+m),r(2*n+m);
    for (j=mm-2;j>=0;j--) {
        fv *= BI;
        fv += v[j];
    }
    fu=1.0/fv;
    for (j=0;j<n;j++) {
        i=Int(fu);
        u[j]=Uchar(i);
        fu=256.0*(fu-i);
    }
    for (;;) {
        mpmul(s,u,v);
        mplsh(s);
        mpneg(s);
        s[0] += Uchar(2);
        mpmul(r,s,u);
        mplsh(r);
        mpmov(u,r);
        for (j=1;j<n-1;j++)
            if (s[j] != 0) break;
        if (j==n-1) return;
    }
}
```

Use ordinary floating arithmetic to get an initial approximation.

Iterate Newton's rule to convergence.

Construct $2 - UV$ in S.

Multiply SU into U.

If fractional part of S is not zero, it has not converged to 1.

Division now follows as a simple corollary, with only the necessity of calculating the reciprocal to sufficient accuracy to get an exact quotient and remainder.

mparith.h

```
void MParith::mpdiv(VecUchar_O &q, VecUchar_O &r, VecUchar_I &u, VecUchar_I &v) {
```

Divides unsigned radix 256 integers `u[0..n-1]` by `v[0..m-1]` (with `m` $\leq$ `n` required), yielding a quotient `q[0..n-m]` and a remainder `r[0..m-1]`.

```
    const Int MACC=1;
    Int i,is,mm,n=u.size(),m=v.size(),p=r.size(),n_min=MIN(m,p);
    if (m > n) throw("Divisor longer than dividend in mpdiv");
    mm=m+MACC;
    VecUchar s(mm),rr(mm),ss(mm+1),qq(n-m+1),t(n);
    mpinv(s,v);
    mpmul(rr,s,u);
    mpsad(ss,rr,1);
```

Set $S = 1/V$.

Set $Q = SU$.

```
    mplsh(ss);
    mplsh(ss);
    mpmov(qq,ss);
    mpmov(q,qq);
    mpmul(t,qq,v);                    Multiply and subtract to get the remainder.
    mplsh(t);
    mpsub(is,t,u,t);
    if (is != 0) throw("MACC too small in mpdiv");
    for (i=0;i<n_min;i++) r[i]=t[i+n-m];
    if (p>m) for (i=m;i<p;i++) r[i]=0;
}
```

Square roots are calculated by a Newton's rule much like division. If

$$U_{i+1} = \tfrac{1}{2}U_i(3 - VU_i^2) \tag{22.7.5}$$

then U_∞ converges quadratically to $1/\sqrt{V}$. A final multiplication by V gives $\sqrt{V}$.

```
void MParith::mpsqrt(VecUchar_O &w, VecUchar_O &u, VecUchar_I &v) {                mparith.h
```

Character string `v[0..m-1]` is interpreted as a radix 256 number with the radix point after `v[0]`; `w[0..n-1]` is set to its square root (radix point after `w[0]`), and `u[0..n-1]` is set to the reciprocal thereof (radix point before `u[0]`). `w` and `u` need not be distinct, in which case they are set to the square root.

```
    const Int MF=3;
    const Doub BI=1.0/256.0;
    Int i,ir,j,n=u.size(),m=v.size(),mm=MIN(m,MF);
    VecUchar r(2*n),x(n+m),s(2*n+m),t(3*n+m);
    Doub fu,fv=Doub(v[mm-1]);
    for (j=mm-2;j>=0;j--) {           Use ordinary floating arithmetic to get an initial ap-
        fv *= BI;                         proximation.
        fv += v[j];
    }
    fu=1.0/sqrt(fv);
    for (j=0;j<n;j++) {
        i=Int(fu);
        u[j]=Uchar(i);
        fu=256.0*(fu-i);
    }
    for (;;) {                        Iterate Newton's rule to convergence.
        mpmul(r,u,u);                 Construct S = (3 − VU^2)/2.
        mplsh(r);
        mpmul(s,r,v);
        mplsh(s);
        mpneg(s);
        s[0] += Uchar(3);
        mpsdv(s,s,2,ir);
        for (j=1;j<n-1;j++) {         If fractional part of S is not zero, it has not con-
            if (s[j] != 0) {              verged to 1.
                mpmul(t,s,u);         Replace U by SU.
                mplsh(t);
                mpmov(u,t);
                break;
            }
        }
        if (j<n-1) continue;
        mpmul(x,u,v);                 Get square root from reciprocal and return.
        mplsh(x);
        mpmov(w,x);
        return;
    }
}
```

We already mentioned that radix conversion to decimal is a merely cosmetic operation that should normally be omitted. The simplest way to convert a fraction to decimal is to multiply it repeatedly by 10, picking off (and subtracting) the resulting integer part. This has an operations count of $O(N^2)$, however, since each liberated decimal digit takes an $O(N)$ operation. It *is* possible to do the radix conversion as a fast operation by a "divide-and-conquer" strategy, in which the fraction is (fast) multiplied by a large power of 10, enough to move about half the desired digits to the left of the radix point. The integer and fractional pieces are now processed independently, each further subdivided. If our goal were a few billion digits of π, instead of a few thousand, we would need to implement this scheme. For present purposes, the following lazy routine is adequate:

mparith.h

```
void MParith::mp2dfr(VecUchar_IO &a, string &s)
```

Converts a radix 256 fraction `a[0..n-1]` (radix point before `a[0]`) to a decimal fraction represented as an ASCII string `s[0..m-1]`, where `m` is a returned value. The input array `a[0..n-1]` is destroyed. NOTE: For simplicity, this routine implements a slow ($\propto N^2$) algorithm. Fast ($\propto N \ln N$), more complicated, radix conversion algorithms do exist.

```
{
    const Uint IAZ=48;
    char buffer[4];
    Int j,m;

    Int n=a.size();
    m=Int(2.408*n);
    sprintf(buffer,"%d",a[0]);
    s=buffer;
    s += '.';
    mplsh(a);
    for (j=0;j<m;j++) {
        mpsmu(a,a,10);
        s += a[0]+IAZ;
        mplsh(a);
    }
}
```

Finally, then, we arrive at a routine implementing equations (22.7.1) and (22.7.2):

mparith.h

```
string MParith::mppi(const Int np) {
```

Demonstrate multiple precision routines by calculating and printing the first `np` bytes of π.

```
    const Uint IAOFF=48,MACC=2;
    Int ir,j,n=np+MACC;
    Uchar mm;
    string s;
    VecUchar x(n),y(n),sx(n),sxi(n),z(n),t(n),pi(n),ss(2*n),tt(2*n);
    t[0]=2;                                  Set T = 2.
    for (j=1;j<n;j++) t[j]=0;
    mpsqrt(x,x,t);                           Set X_0 = √2.
    mpadd(pi,t,x);                           Set π_0 = 2 + √2.
    mplsh(pi);
    mpsqrt(sx,sxi,x);                        Set Y_0 = 2^{1/4}.
    mpmov(y,sx);
    for (;;) {
        mpadd(z,sx,sxi);                     Set X_{i+1} = (X_i^{1/2} + X_i^{-1/2})/2.
        mplsh(z);
        mpsdv(x,z,2,ir);
        mpsqrt(sx,sxi,x);                    Form the temporary T = Y_i X_{i+1}^{1/2} + X_{i+1}^{-1/2}.
        mpmul(tt,y,sx);
        mplsh(tt);
```

```
            mpadd(tt,tt,sxi);
            mplsh(tt);
            x[0]++;                          Increment X_{i+1} and Y_i by 1.
            y[0]++;
            mpinv(ss,y);                     Set Y_{i+1} = T/(Y_i + 1).
            mpmul(y,tt,ss);
            mplsh(y);
            mpmul(tt,x,ss);                  Form temporary T = (X_{i+1} + 1)/(Y_i + 1).
            mplsh(tt);
            mpmul(ss,pi,tt);                 Set π_{i+1} = T π_i.
            mplsh(ss);
            mpmov(pi,ss);
            mm=tt[0]-1;                      If T = 1, then we have converged.
            for (j=1;j < n-1;j++)
                if (tt[j] != mm) break;
            if (j == n-1) {
                mp2dfr(pi,s);
                Convert to decimal for printing. NOTE: The conversion routine, for this demon-
                stration only, is a slow (∝ N^2) algorithm. Fast (∝ N ln N), more complicated,
                radix conversion algorithms do exist.
                s.erase(Int(2.408*np),s.length());
                return s;
            }
        }
    }
```

Figure 22.7.1 gives the result, computed with n = 1000. As an exercise, you might enjoy checking the first hundred digits of the figure against the first 12 terms of Ramanujan's celebrated identity [3]

$$\frac{1}{\pi} = \frac{\sqrt{8}}{9801} \sum_{n=0}^{\infty} \frac{(4n)!\,(1103 + 26390n)}{(n!\,396^n)^4} \tag{22.7.6}$$

using the above routines. You might also use the routines to verify that the number $2^{512} + 1$ is not a prime, but has factors 2,424,833 and 7,455,602,825,647,884,208,337,395,736,200,454,918,783,366,342,657 (which are in fact prime; the remaining prime factor being about 7.416×10^{98}) [4].

CITED REFERENCES AND FURTHER READING:

Borwein, J.M., and Borwein, P.B. 1987, *Pi and the AGM: A Study in Analytic Number Theory and Computational Complexity* (New York: Wiley).[1]

Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §4.3.[2]

Ramanujan, S. 1927, *Collected Papers of Srinivasa Ramanujan*, G.H. Hardy, P.V. Seshu Aiyar, and B.M. Wilson, eds. (Cambridge, UK: Cambridge University Press), pp. 23–39.[3]

Kolata, G. 1990, June 20, "Biggest Division a Giant Leap in Math," *The New York Times*.[4]

Kronsjö, L. 1987, *Algorithms: Their Complexity and Efficiency*, 2nd ed. (New York: Wiley).

3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706798214808651328230664709384460955058223172535940812848111745028410270193852110555964462294895493038196442881097566593344612847564823378678316527120190914564856692346034861045432664821339360726024914127372458700660631558817488152092096282925409171536436789259036001133053054882046652138414695194151160943305727036575959195309218611738193261179310511854807446237996274956735188575272489122793818301194912983367336244065664308602139494639522473719070217986094370277053921717629317675238467481846766940513200056812714526356082778577134275778960917363717872146844090122495343014654958537105079227968925892354201995611212902196086403441815981362977477130996051870721134999999837297804995105973173281609631859502445945534690830264252230825334468503526193118817101000313783875288658753320838142061717766914730359825349042875546873115956286388235378759375195778185778053217122680661300192787661119590921642019893809525720106548586327886593615338182796823030195203530185296899577362259941389124972177528347913151557485724245415069595082953311686172785588907509838175463746493931925506040092770167113900984882401285836160356370766010471018194295559619894676783744944825537977472684710404753464620804668425906949129331367702898915210475216205696602405803815019351125338243003558764024749647326391419927260426992279678235478163600934172164121992458631503028618297455570674983850549458858692699569092721079750930295532116534498720275596023648066549911988183479775356636980742654252786255181841757467289097777279380008164706001614524919217321721477235014144197356854816136115735255213347574184946843852332390739414333454776241686251898356948556209921922218427255025425688767179049460165346680498862723279178608578438382796797668145410095388378636095068006422512520511739298489608412848862694560424196528502221066118630674427862203919494504712371378696095636437191728746765757396241389086583264599581339047802759009946576407895126946839835259570982582262052248940772671947826848260147699090264013639443745530506820349625245174939965143142980919065925093722169646151570985838741059785895977297549893016175392846813826868386894277415599185592524595395943104997252468084598727364469584865383673622262609912460805124388439045124413654976278079771569143599770012961608944169486855584840635342207222582848864815845602859 3.

Figure 22.7.1. The first 2398 decimal digits of π, computed by the routines in this section.